

|              |                                 |
|--------------|---------------------------------|
| Started on   | Saturday, 21 June 2025, 4:53 PM |
| State        | Finished                        |
| Completed on | Saturday, 21 June 2025, 5:01 PM |
| Time taken   | 8 mins 27 secs                  |
| Marks        | 2.00/2.00                       |
| Grade        | 10.00 out of 10.00 (100%)       |

Viết chương trình đọc một **đơn đồ thị có hướng, có trọng số không âm** từ bàn phím, tìm đường đi ngắn nhất từ đỉnh s đến đỉnh t (s và t cũng được đọc từ bàn phím).

### Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m ( $1 \leq n < 100; 0 \leq m < 500$ )
- m dòng tiếp theo mỗi dòng chứa 3 số nguyên u, v, w mô tả cung (u, v) có trọng số w ( $0 \leq w \leq 100$ ).
- Dòng cuối cùng chứa 2 số nguyên s và t.

### Đầu ra (Output)

- In đường đi ngắn nhất từ s đến t theo mẫu:

```
s -> u1 -> u2 -> ... -> t
```

- Xem thêm ví dụ bên dưới.

### Gợi ý

- Lần ngược theo p[u] để có được đường đi ngắn nhất.

For example:

| Input                                  | Result      |
|----------------------------------------|-------------|
| 3 3<br>1 2 3<br>2 3 5<br>1 3 10<br>1 3 | 1 -> 2 -> 3 |

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 101
4
5 //-----Graph-----
6 #define NO_EDGE -1
7 typedef struct {
8     int n, m;
9     int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 void init_graph(Graph* pG, int n) {
13     pG->n = n;
14     pG->m = 0;
15
16     for (int i = 0; i <= n; i++) {
17         for (int j = 0; j <= n; j++) {
18             pG->A[i][j] = NO_EDGE;
19         }
20     }
21 }
22
23 void add_edge(Graph* pG, int u, int v, int w) {
24     pG->A[u][v] = w;
25
26     pG->m++;
27 }
28
29 int adj(Graph* pG, int u, int v) {
30     return pG->A[u][v] > NO_EDGE;
31 }
32
33 //-----Dijkstra path-----
34 #define NOT_SURE 0
35 #define SURE 1
36 #define oo 999999
37
38 int parent[MAX_SIZE];
39 int visited[MAX_SIZE];
40 int d[MAX_SIZE];
```

```

41 void Dijkstra(Graph* pG, int s, int t) {
42     // 1. init
43     for (int i = 1; i <= pG->n; i++) {
44         d[i] = oo;
45         visited[i] = NOT_SURE;
46     }
47
48     d[s] = 0;
49     parent[s] = -1;
50
51     int u = -1;
52
53     // duyet cac dinh
54     for (int i = 1; i <= pG->n; i++) {
55         // 2. tim min
56         int min_u = oo;
57         for (int v = 1; v <= pG->n; v++) {
58             if (visited[v] == NOT_SURE && d[v] < min_u) {
59                 min_u = d[v];
60                 u = v;
61             }
62         }
63
64         if (u == -1) break;
65         visited[u] = SURE;
66         if (u == t) break;
67
68         // 3. cap nhat cac dinh ke
69         for (int v = 1; v <= pG->n; v++) {
70             int w = pG->A[u][v];
71             if (adj(pG, u, v) && (d[u] + w < d[v])) {
72                 d[v] = d[u] + w;
73                 parent[v] = u;
74             }
75         }
76     }
77
78     // 4. in duong di
79     int k = 0;
80     int path[MAX_SIZE];
81
82     int current = u;
83
84     // while (current != parent[s]) {
85     //     path[k] = current;
86     //     current = parent[current];
87     //     k++;
88     // }
89
90     while (current != s) {
91         // printf("%d ", current);
92         path[k] = current;
93         k++;
94         current = parent[current];
95     }
96
97     path[k] = s;
98
99     printf("%d ", path[k]);
100
101     for (int i = k - 1; i >= 0; i--) {
102         printf("-> %d ", path[i]);
103     }
104     // printf("%d ", path[k - 1]);
105     // for (int i = k - 2; i >= 0; i--) {
106     //     printf("-> %d ", path[i]);
107     // }
108 }
109
110 int main() {
111     Graph G;
112     int n, m;
113     scanf("%d%d", &n, &m);
114     init_graph(&G, n);
115
116     for (int e = 0; e < m; e++) {
117         int

```

```

119     int u, v, w;
120     scanf("%d%d%d", &u, &v, &w);
121     add_edge(&G, u, v, w);
122 }
123
124 int s, t;
125 scanf("%d %d", &s, &t);
126 Dijkstra(&G, s, t);
127 // for (int u = 1; u <= G.n; u++) {
128 //     printf("pi[%d] = %d, p[%d] = %d\n", u, d[u], u, parent[u]);
129 // }
130 }
131

```

|   | Input                                                                                           | Expected         | Got              |   |
|---|-------------------------------------------------------------------------------------------------|------------------|------------------|---|
| ✓ | 3 3<br>1 2 3<br>2 3 5<br>1 3 10<br>1 3                                                          | 1 -> 2 -> 3      | 1 -> 2 -> 3      | ✓ |
| ✓ | 3 3<br>1 3 5<br>3 2 3<br>1 2 10<br>1 2                                                          | 1 -> 3 -> 2      | 1 -> 3 -> 2      | ✓ |
| ✓ | 6 9<br>1 2 7<br>1 3 9<br>1 5 14<br>2 3 10<br>2 4 15<br>3 4 11<br>3 5 2<br>4 6 6<br>5 6 9<br>1 5 | 1 -> 3 -> 5      | 1 -> 3 -> 5      | ✓ |
| ✓ | 6 9<br>1 2 7<br>1 3 9<br>1 5 14<br>2 3 10<br>2 4 15<br>3 4 11<br>3 5 2<br>4 6 6<br>5 6 9<br>2 6 | 2 -> 3 -> 5 -> 6 | 2 -> 3 -> 5 -> 6 | ✓ |
| ✓ | 6 9<br>1 2 7<br>1 3 9<br>1 5 14<br>2 3 10<br>2 4 15<br>3 4 11<br>3 5 3<br>4 6 6<br>6 5 9<br>2 5 | 2 -> 3 -> 5      | 2 -> 3 -> 5      | ✓ |

Passed all tests! ✓

Question author's solution (C):

```

1 #include <stdio.h>
2

```

```

3 #define MAXN 100
4 #define oo 999999
5 #define NO_EDGE -1
6
7
8 typedef struct {
9     int n, m;
10    int W[MAXN][MAXN];
11 } Graph;
12
13 void init_graph(Graph *pG, int n) {
14     pG->n = n;
15     pG->m = 0;
16     for (int u = 1; u <= n; u++)
17         for (int v = 1; v <= n; v++)
18             pG->W[u][v] = NO_EDGE;
19 }
20
21 void add_edge(Graph *pG, int u, int v, int w) {
22     pG->W[u][v] = w;
23 }
24
25 int mark[MAXN];
26 int pi[MAXN];
27 int p[MAXN];
28
29 void MooreDijkstra(Graph *pG, int s) {
30     int u, v, it;
31     for (u = 1; u <= pG->n; u++) {
32         pi[u] = oo;
33         mark[u] = 0;
34     }
35
36     pi[s] = 0; //chiều dài đường đi ngắn nhất từ s đến chính nó bằng 0
37     p[s] = -1; //trước đỉnh s không có đỉnh nào cả
38
39     //Lặp n-1 lần
40     for (it = 1; it < pG->n; it++) {
41         //1. Tìm u có mark[u] == 0 và có pi[u] nhỏ nhất
42         int j, min_pi = oo;
43         for (j = 1; j <= pG->n; j++)
44             if (mark[j] == 0 && pi[j] < min_pi) {
45                 min_pi = pi[j];
46                 u = j;
47             }
48         //2. Đánh dấu u đã xét
49         mark[u] = 1;
50
51         //3. Cập nhật pi và p của các đỉnh kề của u (nếu thoả)
52         for (v = 1; v <= pG->n; v++)
53             if (pG->W[u][v] != NO_EDGE && mark[v] == 0)
54                 if (pi[u] + pG->W[u][v] < pi[v]) {
55                     pi[v] = pi[u] + pG->W[u][v];
56                     p[v] = u;
57                 }
58     }
59 }
60
61
62 int main() {
63     Graph G;
64     int n, m;
65     scanf("%d%d", &n, &m);
66     init_graph(&G, n);
67
68     for (int e = 0; e < m; e++) {
69         int u, v, w;
70         scanf("%d%d%d", &u, &v, &w);
71         add_edge(&G, u, v, w);
72     }
73
74     int s, t;
75     scanf("%d%d", &s, &t);
76
77     MooreDijkstra(&G, s);
78
79     //Tìm đường đi ngắn nhất
80     int path[MAXN]; //lưu các đỉnh trên đường đi

```

```
81     int k = 0;           //số đỉnh của đường đi
82     int current = t;
83
84     //Lần ngược theo p để lấy đường đi
85     while (current != -1) {
86         path[k] = current; k++;
87         current = p[current];
88     }
89
90     //In ra màn hình theo chiều ngược lại
91     printf("%d", path[k-1]);
92
93     for (int u = k-2; u >= 0; u--)
94         printf(" -> %d", path[u]);
95
96
97
98     return 0;
99 }
```

**Correct**

Marks for this submission: 1.00/1.00.



Viết chương trình đọc một **đơn đồ thị vô hướng, có trọng số không âm** từ bàn phím, tìm đường đi ngắn nhất từ đỉnh s đến đỉnh t (s và t cũng được đọc từ bàn phím).

### Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m ( $1 \leq n < 100$ ;  $0 \leq m < 500$ )
- m dòng tiếp theo mỗi dòng chứa 3 số nguyên u, v, w mô tả cung (u, v) có trọng số w ( $0 \leq w \leq 100$ ).
- Dòng cuối cùng chứa 2 số nguyên s và t.

### Đầu ra (Output)

- In đường đi ngắn nhất từ s đến t theo mẫu:

```
s -> u1 -> u2 -> ... -> t
```

- Xem thêm ví dụ bên dưới.

### Gợi ý

- Lần ngược theo p[u] để có được đường đi ngắn nhất.

For example:

| Input                                  | Result      |
|----------------------------------------|-------------|
| 3 3<br>1 2 3<br>2 3 5<br>1 3 10<br>3 1 | 3 -> 2 -> 1 |

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 101
4
5 //-----Graph-----
6 #define NO_EDGE -1
7 typedef struct {
8     int n, m;
9     int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 void init_graph(Graph* pG, int n) {
13     pG->n = n;
14     pG->m = 0;
15
16     for (int i = 0; i <= n; i++) {
17         for (int j = 0; j <= n; j++) {
18             pG->A[i][j] = NO_EDGE;
19         }
20     }
21 }
22
23 void add_edge(Graph* pG, int u, int v, int w) {
24     pG->A[u][v] = w;
25     pG->A[v][u] = w;
26     pG->m++;
27 }
28
29 int adj(Graph* pG, int u, int v) {
30     return pG->A[u][v] > NO_EDGE;
31 }
32
33 //-----Dijkstra path-----
34 #define NOT_SURE 0
35 #define SURE 1
36 #define oo 999999
37
38 int parent[MAX_SIZE];
39 int visited[MAX_SIZE];
40 int d[MAX_SIZE];
```

```

41 void Dijkstra(Graph* pG, int s, int t) {
42     // 1. init
43     for (int i = 1; i <= pG->n; i++) {
44         d[i] = oo;
45         visited[i] = NOT_SURE;
46     }
47
48     d[s] = 0;
49     parent[s] = -1;
50
51     int u = -1;
52
53     // duyet cac dinh
54     for (int i = 1; i <= pG->n; i++) {
55         // 2. tim min
56         int min_u = oo;
57         for (int v = 1; v <= pG->n; v++) {
58             if (visited[v] == NOT_SURE && d[v] < min_u) {
59                 min_u = d[v];
60                 u = v;
61             }
62         }
63
64         if (u == -1) break;
65         visited[u] = SURE;
66         if (u == t) break;
67
68         // 3. cap nhat cac dinh ke
69         for (int v = 1; v <= pG->n; v++) {
70             int w = pG->A[u][v];
71             if (adj(pG, u, v) && (d[u] + w < d[v])) {
72                 d[v] = d[u] + w;
73                 parent[v] = u;
74             }
75         }
76     }
77
78     // 4. in duong di
79     int k = 0;
80     int path[MAX_SIZE];
81
82     int current = u;
83
84     // while (current != parent[s]) {
85     //     path[k] = current;
86     //     current = parent[current];
87     //     k++;
88     // }
89
90     while (current != s) {
91         // printf("%d ", current);
92         path[k] = current;
93         k++;
94         current = parent[current];
95     }
96
97     path[k] = s;
98
99     printf("%d ", path[k]);
100
101     for (int i = k - 1; i >= 0; i--) {
102         printf("-> %d ", path[i]);
103     }
104     // printf("%d ", path[k - 1]);
105     // for (int i = k - 2; i >= 0; i--) {
106     //     printf("-> %d ", path[i]);
107     // }
108 }
109
110 int main() {
111     Graph G;
112     int n, m;
113     scanf("%d%d", &n, &m);
114     init_graph(&G, n);
115
116     for (int e = 0; e < m; e++) {
117         int

```

```

119     int u, v, w;
120     scanf("%d%d%d", &u, &v, &w);
121     add_edge(&G, u, v, w);
122 }
123
124 int s, t;
125 scanf("%d %d", &s, &t);
126 Dijkstra(&G, s, t);
127 // for (int u = 1; u <= G.n; u++) {
128 //     printf("pi[%d] = %d, p[%d] = %d\n", u, d[u], u, parent[u]);
129 // }
130 }
131

```

|   | Input                                                                                           | Expected         | Got              |   |
|---|-------------------------------------------------------------------------------------------------|------------------|------------------|---|
| ✓ | 3 3<br>1 2 3<br>2 3 5<br>1 3 10<br>3 1                                                          | 3 -> 2 -> 1      | 3 -> 2 -> 1      | ✓ |
| ✓ | 3 3<br>1 3 5<br>3 2 3<br>1 2 10<br>2 1                                                          | 2 -> 3 -> 1      | 2 -> 3 -> 1      | ✓ |
| ✓ | 6 9<br>1 2 7<br>1 3 9<br>1 5 14<br>2 3 10<br>2 4 15<br>3 4 11<br>3 5 2<br>4 6 6<br>5 6 9<br>6 2 | 6 -> 4 -> 2      | 6 -> 4 -> 2      | ✓ |
| ✓ | 6 9<br>1 2 7<br>1 3 9<br>1 5 14<br>2 3 10<br>2 4 15<br>3 4 11<br>3 5 2<br>4 6 6<br>5 6 9<br>6 1 | 6 -> 5 -> 3 -> 1 | 6 -> 5 -> 3 -> 1 | ✓ |
| ✓ | 6 9<br>1 2 7<br>1 3 9<br>1 5 14<br>2 3 10<br>2 4 15<br>3 4 11<br>3 5 3<br>4 6 6<br>6 5 9<br>5 2 | 5 -> 3 -> 2      | 5 -> 3 -> 2      | ✓ |

Passed all tests! ✓

Question author's solution (C):

```

1 #include <stdio.h>
2
3 #define MAXN 100
4 #define oo 999999

```

```

4 #define oo 999999
5 #define NO_EDGE -1
6
7
8 typedef struct {
9     int n, m;
10    int W[MAXN][MAXN];
11 } Graph;
12
13 void init_graph(Graph *pG, int n) {
14     pG->n = n;
15     pG->m = 0;
16     for (int u = 1; u <= n; u++)
17         for (int v = 1; v <= n; v++)
18             pG->W[u][v] = NO_EDGE;
19 }
20
21 void add_edge(Graph *pG, int u, int v, int w) {
22     pG->W[u][v] = w;
23     pG->W[v][u] = w;
24     pG->m++;
25 }
26
27 int mark[MAXN];
28 int pi[MAXN];
29 int p[MAXN];
30
31 void MooreDijkstra(Graph *pG, int s) {
32     int u, v, it;
33     for (u = 1; u <= pG->n; u++) {
34         pi[u] = oo;
35         mark[u] = 0;
36     }
37
38     pi[s] = 0; //chiều dài đường đi ngắn nhất từ s đến chính nó bằng 0
39     p[s] = -1; //trước đỉnh s không có đỉnh nào cả
40
41     //Lặp n-1 lần
42     for (it = 1; it < pG->n; it++) {
43         //1. Tìm u có mark[u] == 0 và có pi[u] nhỏ nhất
44         int j, min_pi = oo;
45         for (j = 1; j <= pG->n; j++)
46             if (mark[j] == 0 && pi[j] < min_pi) {
47                 min_pi = pi[j];
48                 u = j;
49             }
50         //2. Đánh dấu u đã xét
51         mark[u] = 1;
52
53         //3. Cập nhật pi và p của các đỉnh kề của u (nếu thoả)
54         for (v = 1; v <= pG->n; v++)
55             if (pG->W[u][v] != NO_EDGE && mark[v] == 0)
56                 if (pi[u] + pG->W[u][v] < pi[v]) {
57                     pi[v] = pi[u] + pG->W[u][v];
58                     p[v] = u;
59                 }
60     }
61 }
62
63
64 int main() {
65     Graph G;
66     int n, m;
67     scanf("%d%d", &n, &m);
68     init_graph(&G, n);
69
70     for (int e = 0; e < m; e++) {
71         int u, v, w;
72         scanf("%d%d%d", &u, &v, &w);
73         add_edge(&G, u, v, w);
74     }
75
76     int s, t;
77     scanf("%d%d", &s, &t);
78
79     MooreDijkstra(&G, s);
80
81     //Tìm đường đi ngắn nhất
82     int path[MAXN]; //lưu các đỉnh trên đường đi

```

```

82     int path[1000]; //lưu các đỉnh trên đường đi
83     int k = 0;      //số đỉnh của đường đi
84     int current = t;
85
86     //Lần ngược theo p để lấy đường đi
87     while (current != -1) {
88         path[k] = current; k++;
89         current = p[current];
90     }
91
92     //In ra màn hình theo chiều ngược lại
93     printf("%d", path[k-1]);
94
95     for (int u = k-2; u >= 0; u--)
96         printf(" -> %d", path[u]);
97
98
99
100     return 0;
101 }

```

**Correct**

Marks for this submission: 1.00/1.00.



◀ Bài tập 2 - Thuật toán Moore -  
Dijkstra (chiều dài)

Jump to...



Bài tập 4\* - Mê cung số ▶

